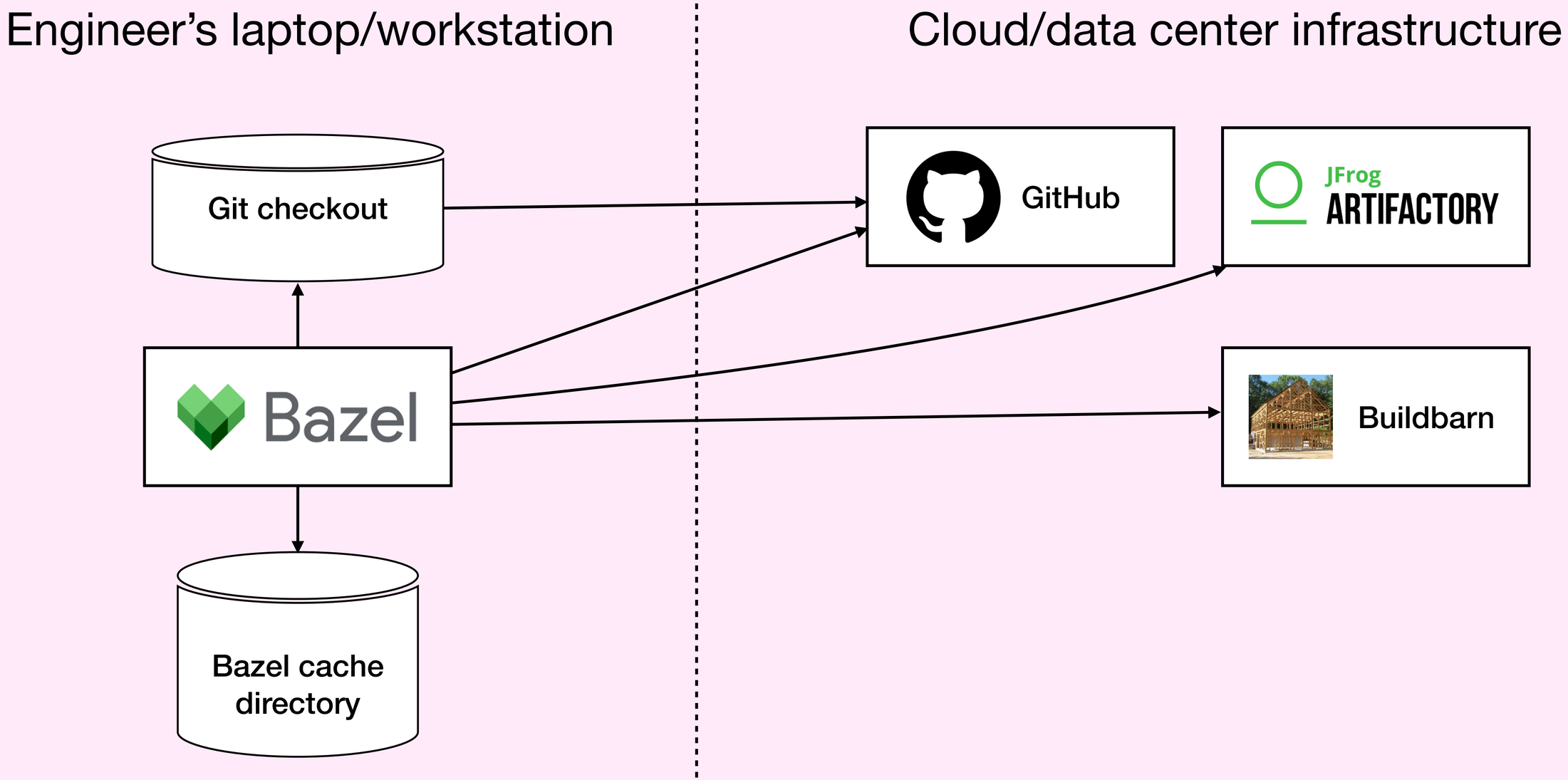
Caching analysis data in Bonanza

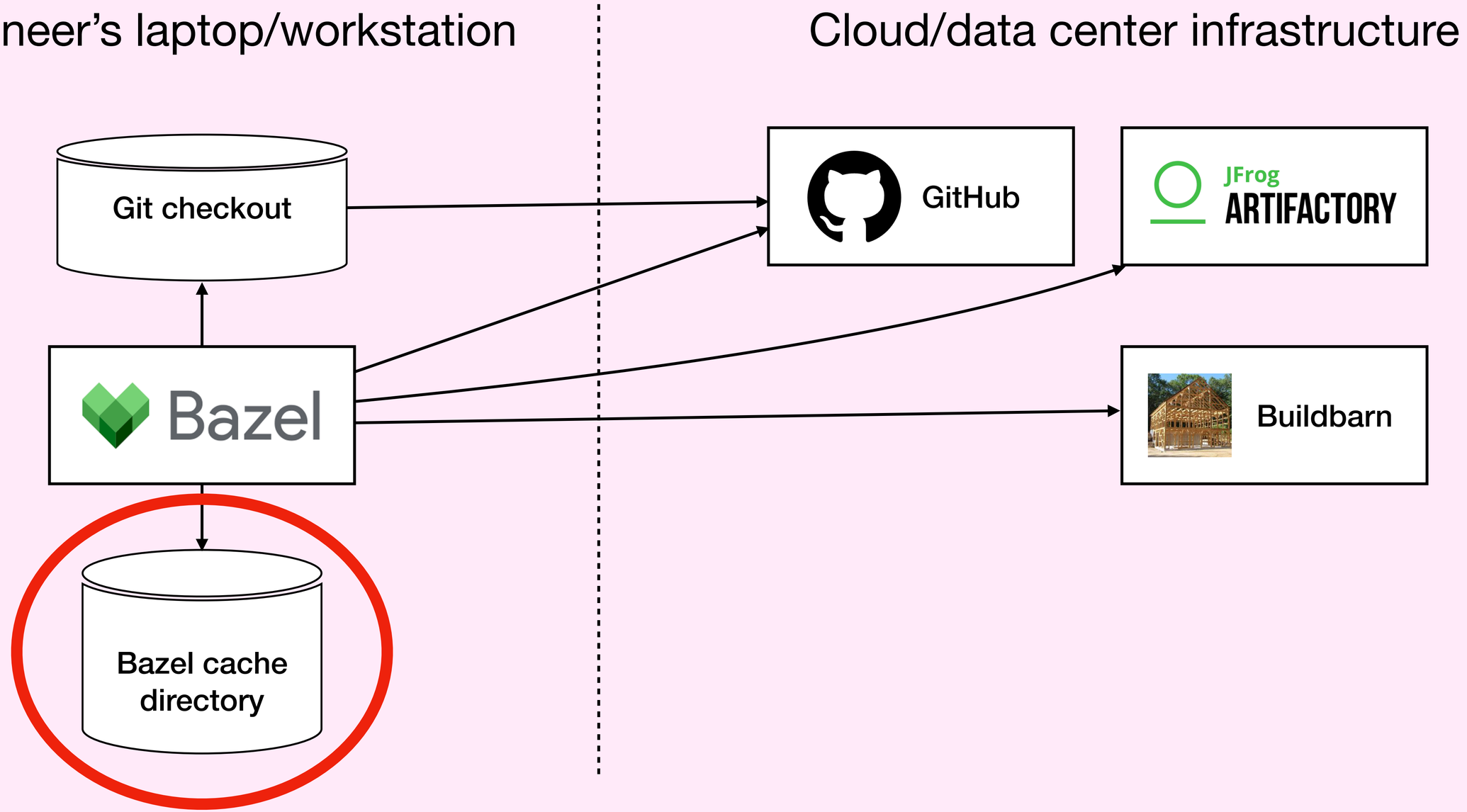
2026-10-13 Buildbarn Meetup at Datadog

Ed Schouten <ed@nuxi.nl>

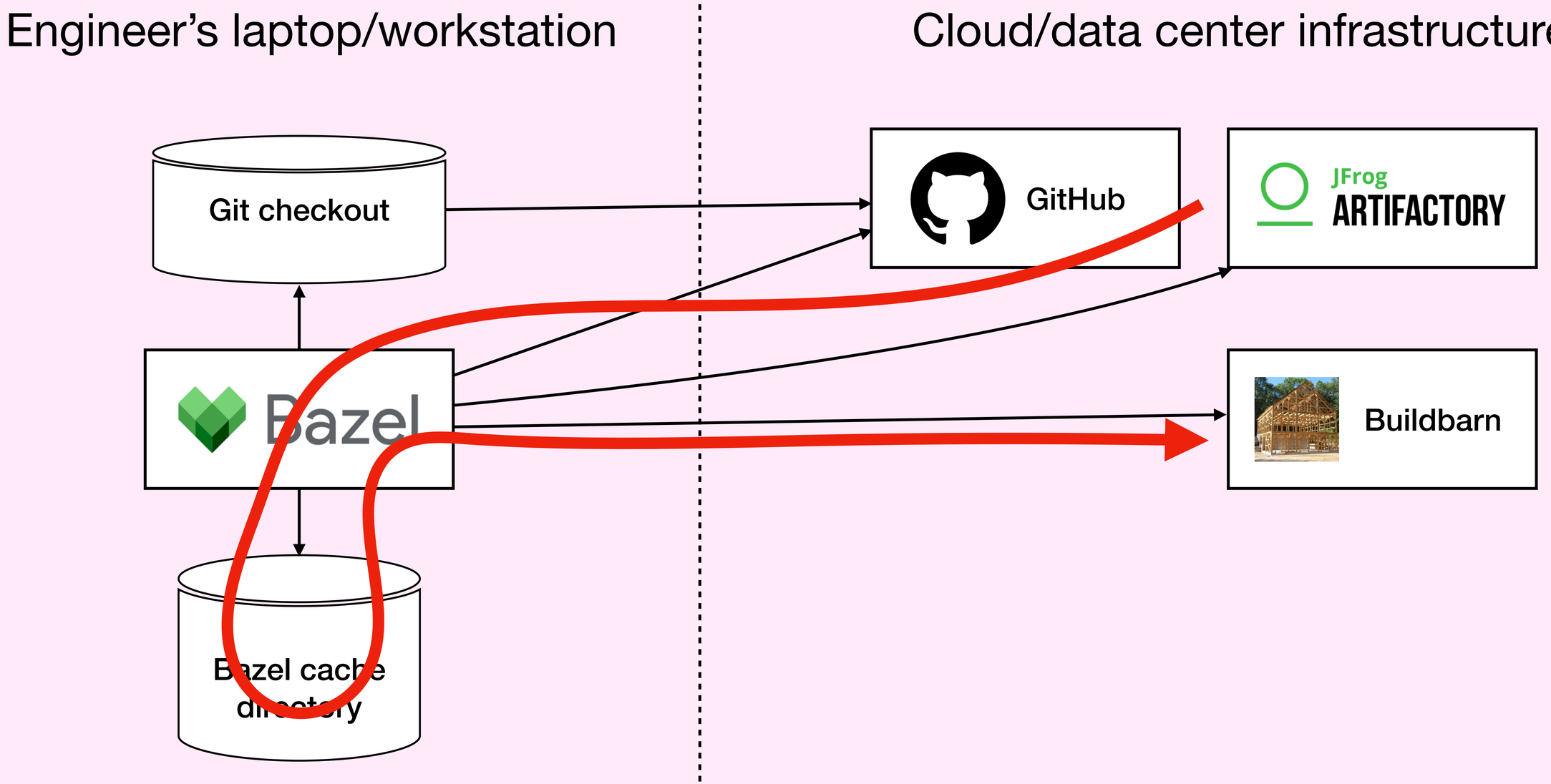
Example of a Buildbarn setup



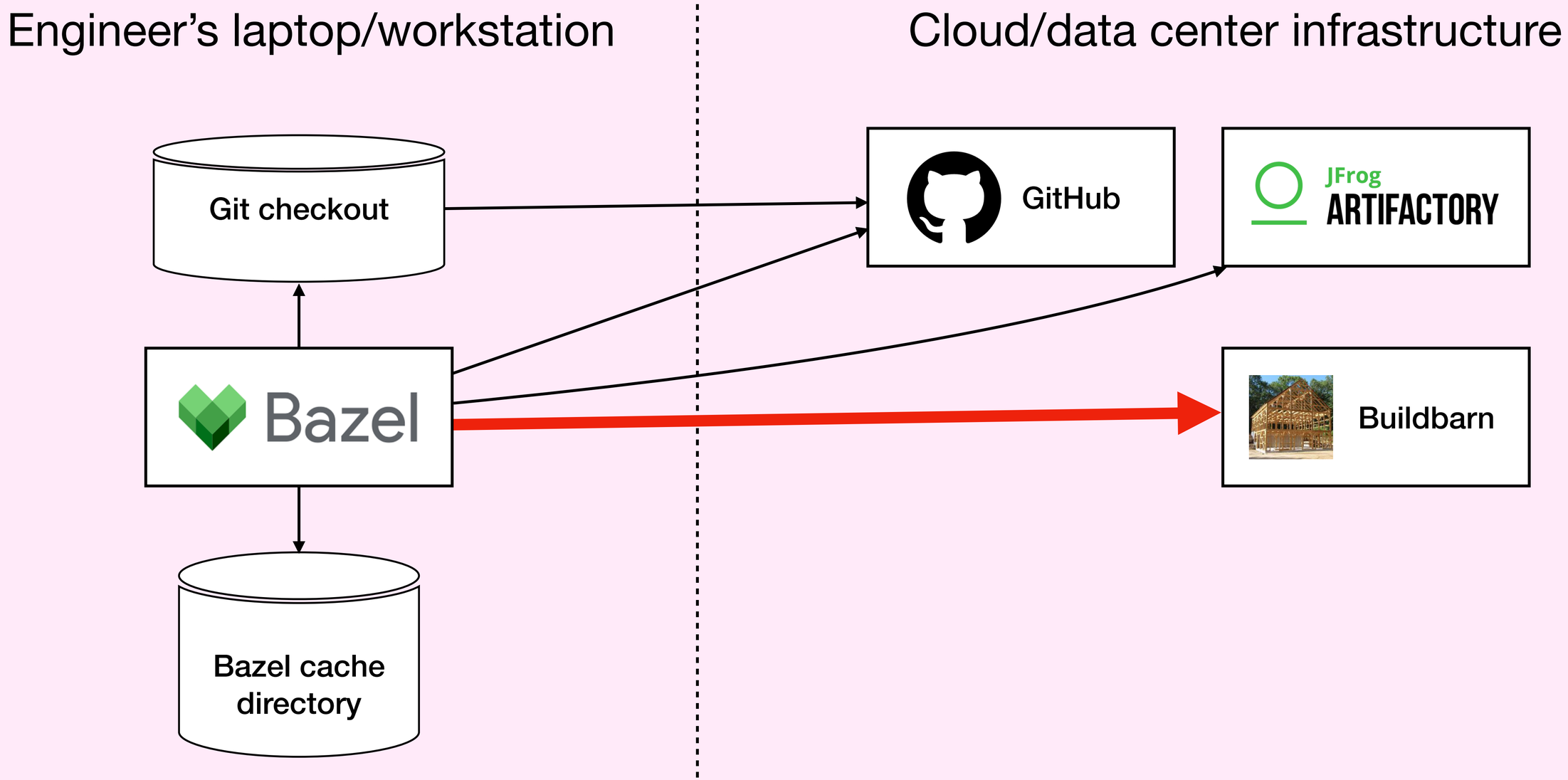
Problem #1: Excessive disk space usage



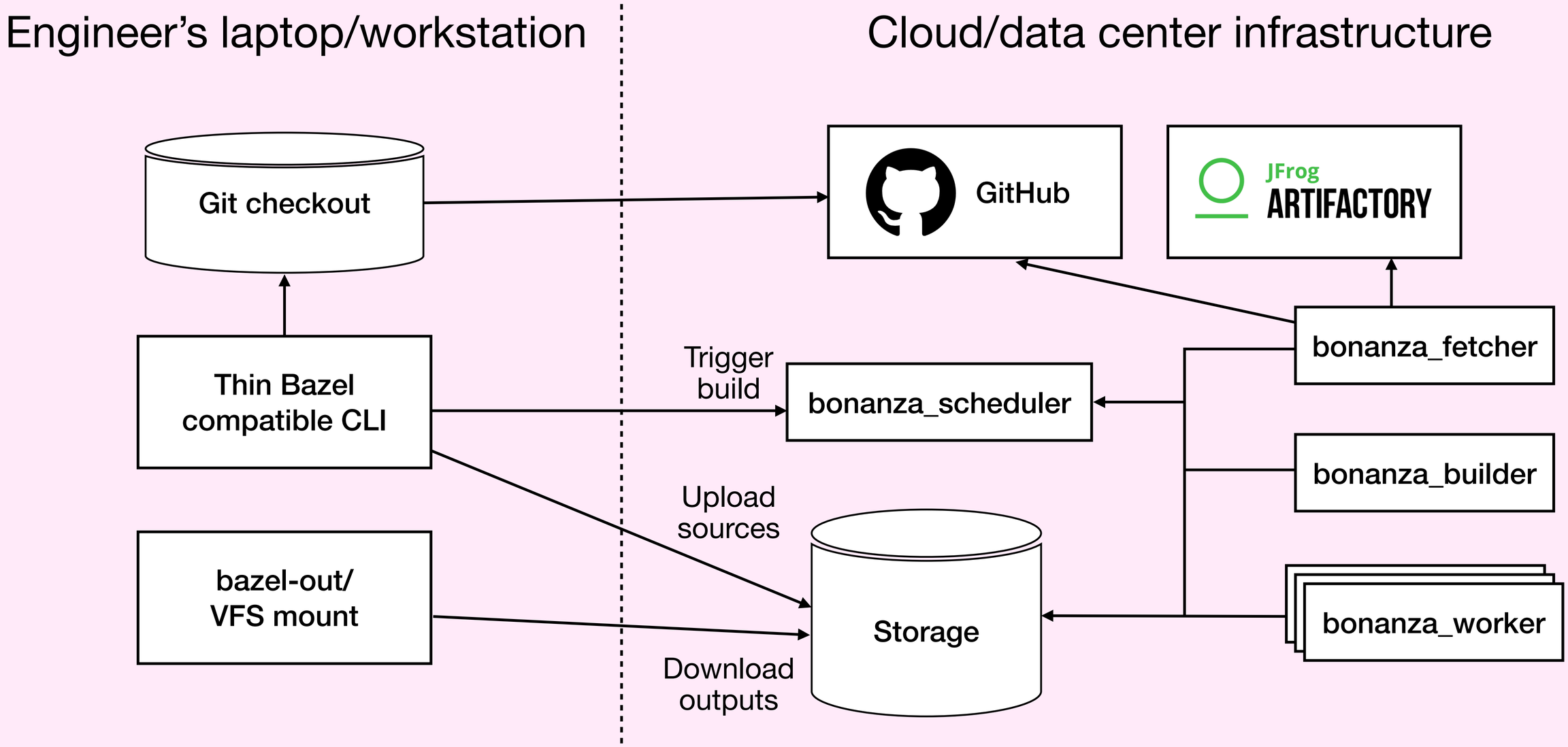
Problem #2: unnecessary flow of assets



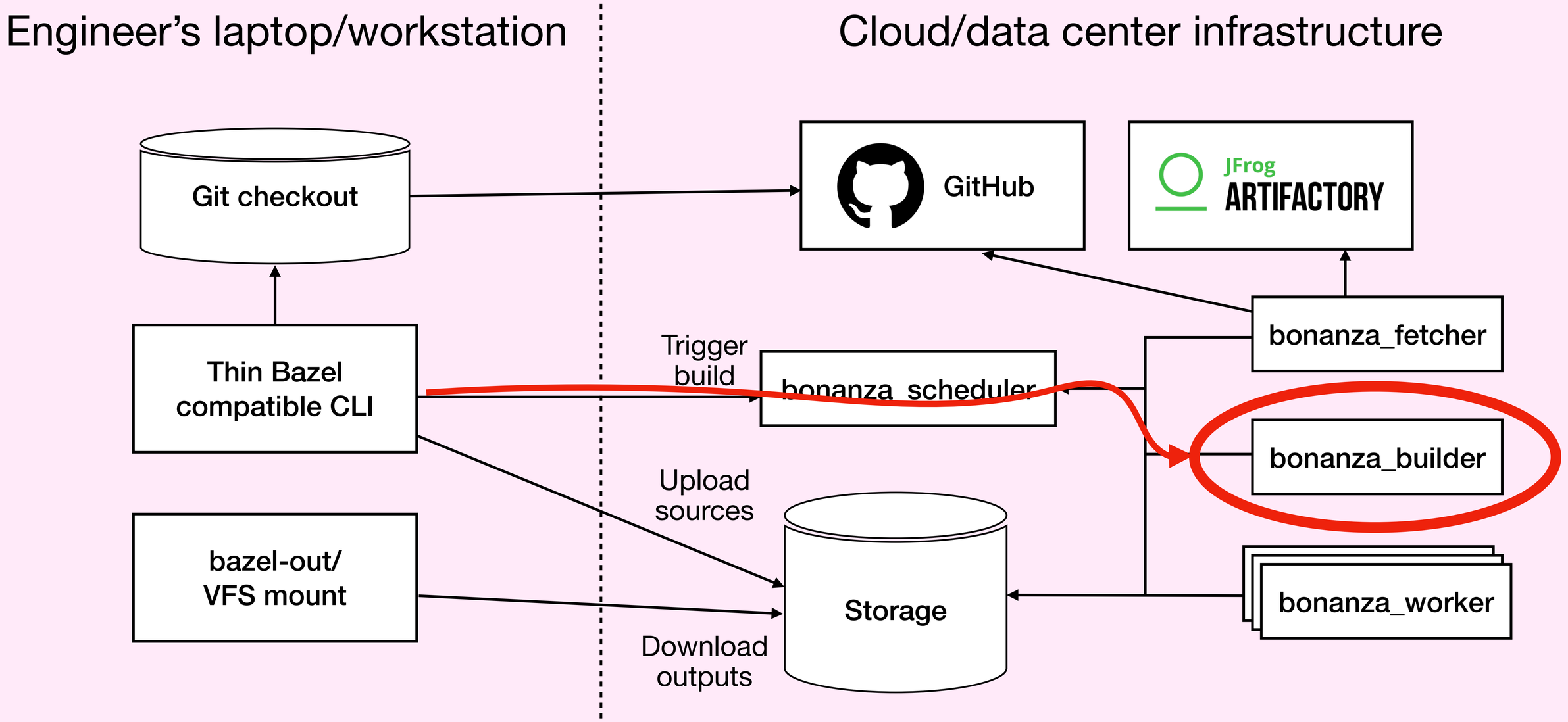
Problem #3: latency sensitivity



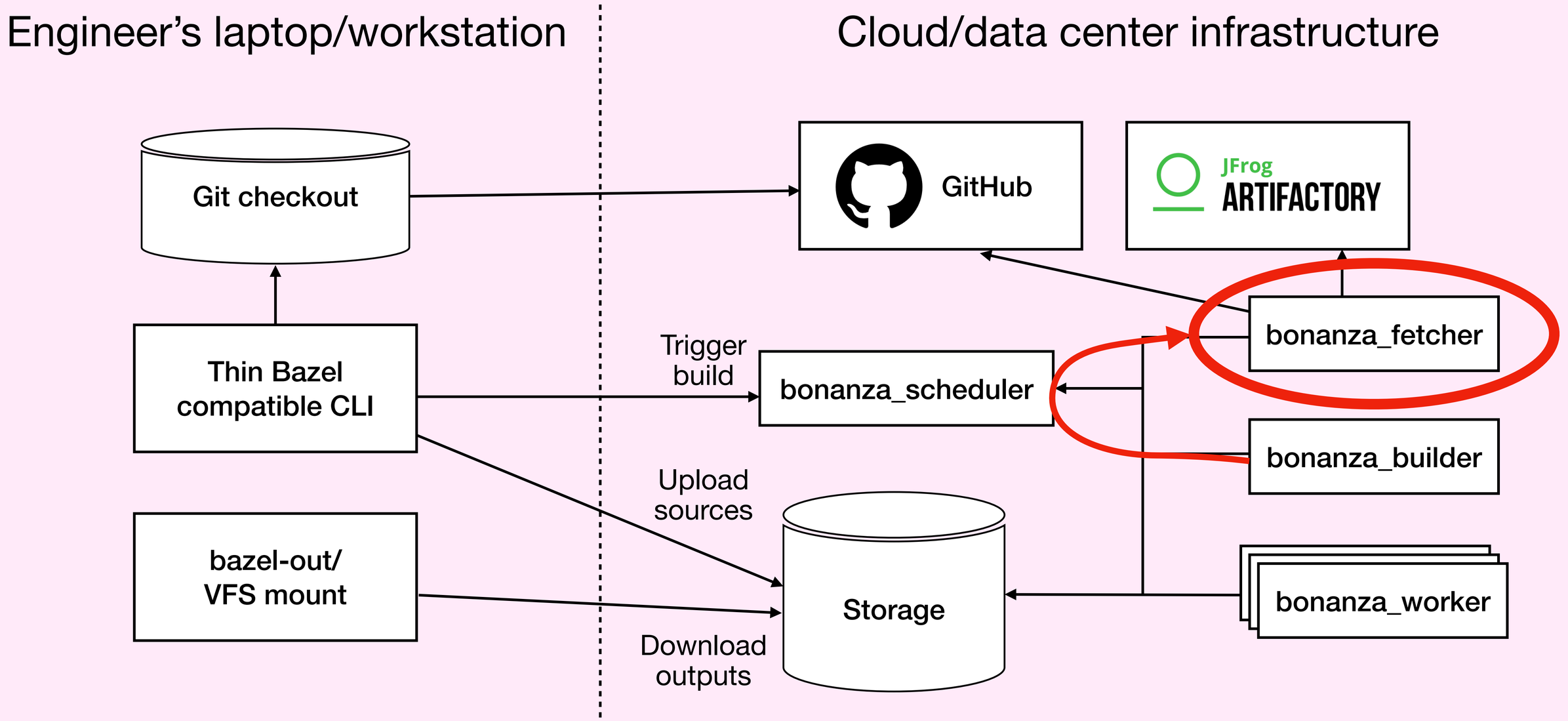
Bonanza: a remote build system



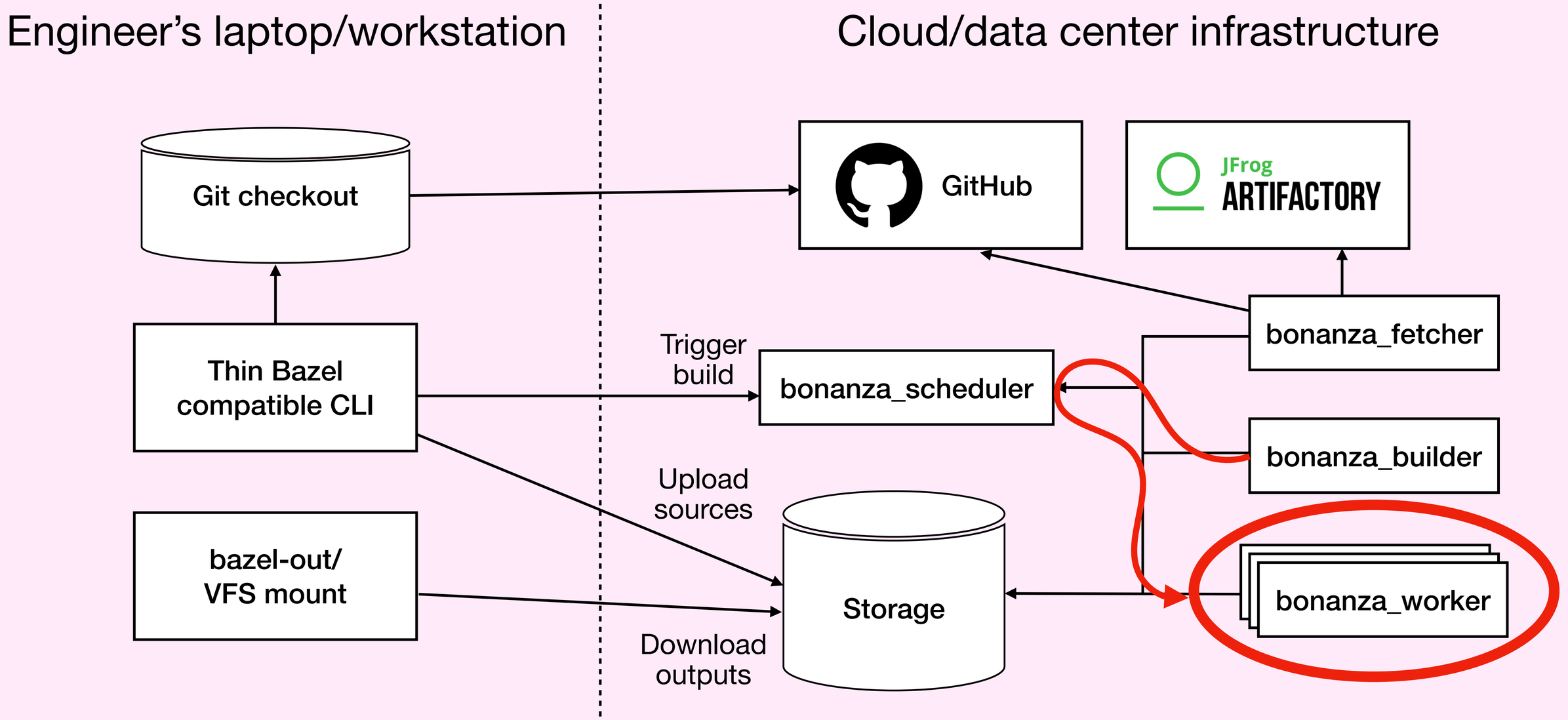
Bonanza: a remote build system



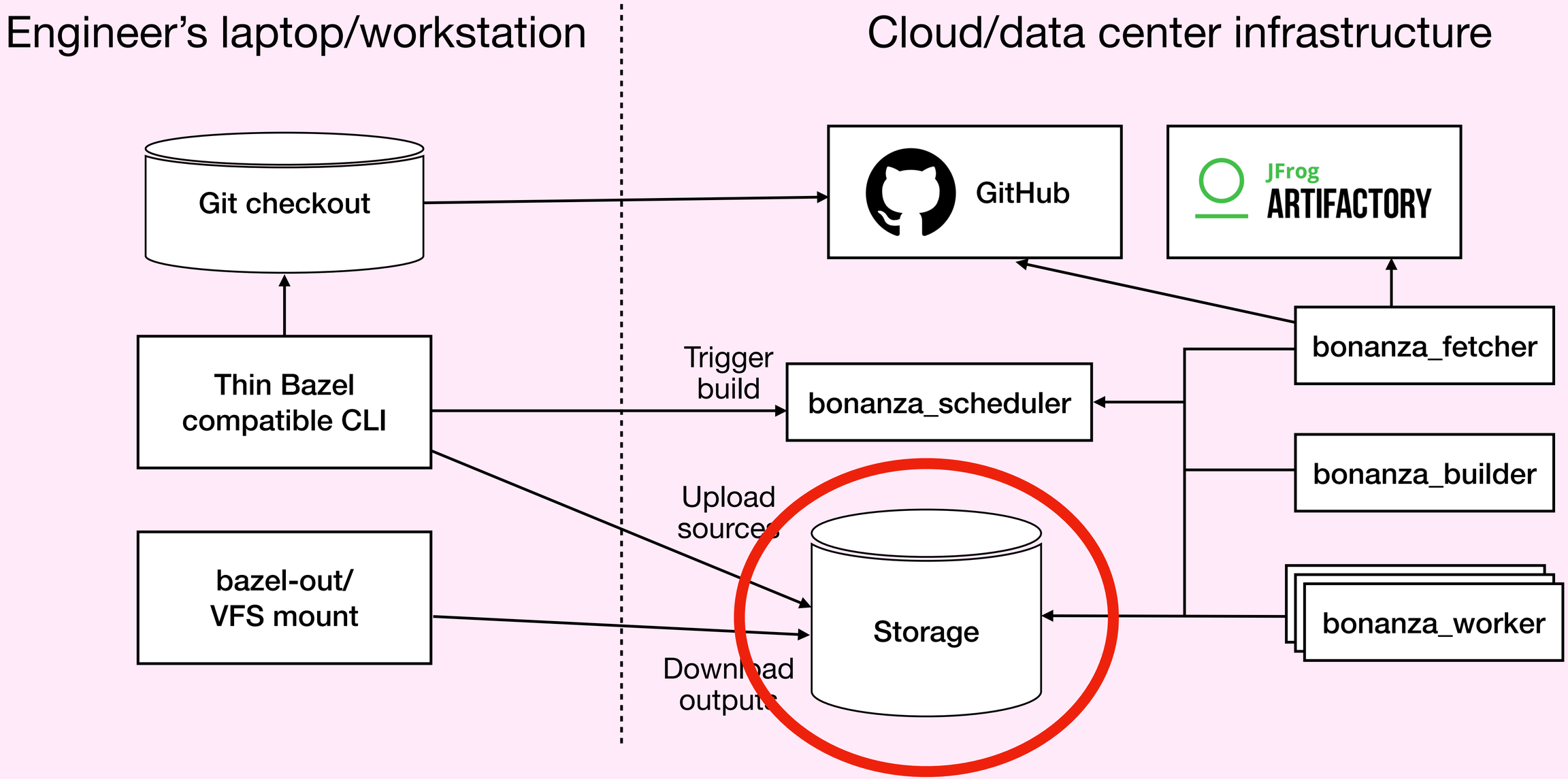
Bonanza: a remote build system



Bonanza: a remote build system



Bonanza: a remote build system



Storage

Buildbarn's data stores

- **CAS:** Content Addressable Storage
- **AC:** Action Cache
- **ISCC:** Initial Size Class Cache
- **FSAC:** File System Access Cache
- **ICAS:** Indirect Content Addressable Storage

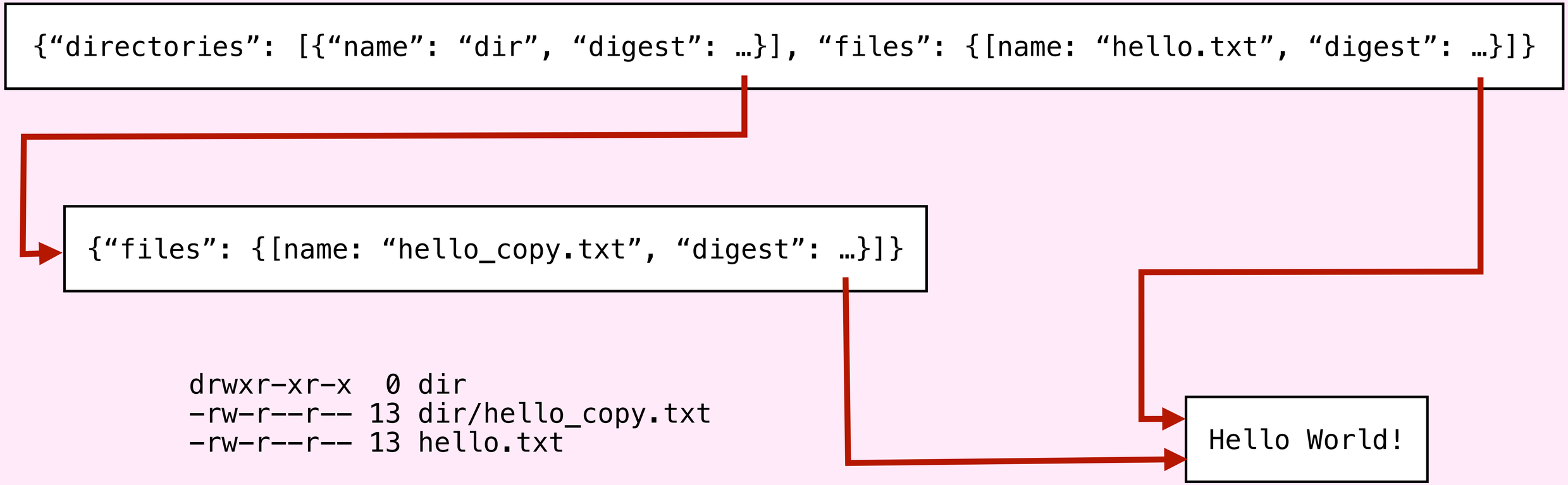
Buildbarn's data stores

- ~~**CAS:** Content Addressable Storage~~
  - **CS:** Chunk Store
  - **CMS:** Chunk Mapping Store
- **AC:** Action Cache
- **ISCC:** Initial Size Class Cache
- **FSAC:** File System Access Cache
- **ICAS:** Indirect Content Addressable Storage

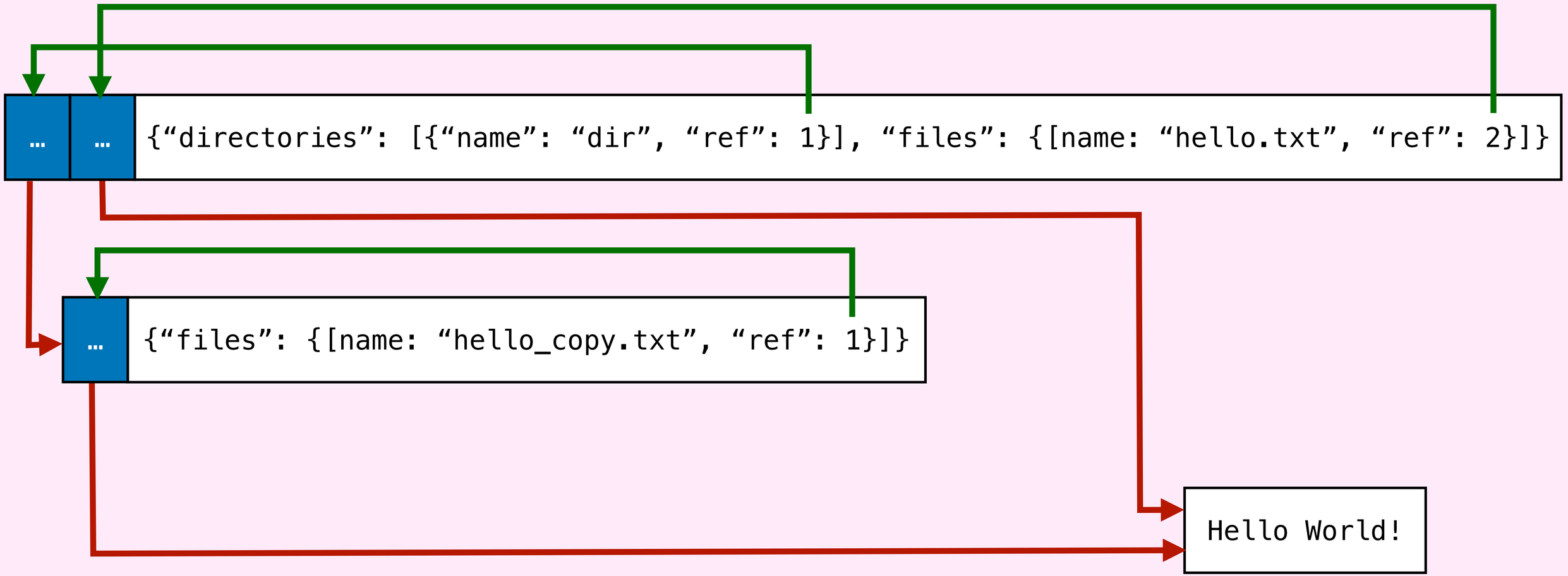
Bonanza’s data stores

- Object Store
  - Replacement for the CAS.
- Tag Store
  - Replacement for everything not-CAS.

REv2 directory hierarchy



Equivalent data in Bonanza Object Store



Advantages of Bonanza's Object Store

- Storage is fully oblivious of the format of the data being stored.
  - Easier to store things other than actions, directories, etc.
  - Easier to optimize data layouts: inline small directories, Prolly trees, etc.
  - Chunking, compression and encryption are added as layers on top.
- Will permit smarter graph based garbage collection.

Tag Store

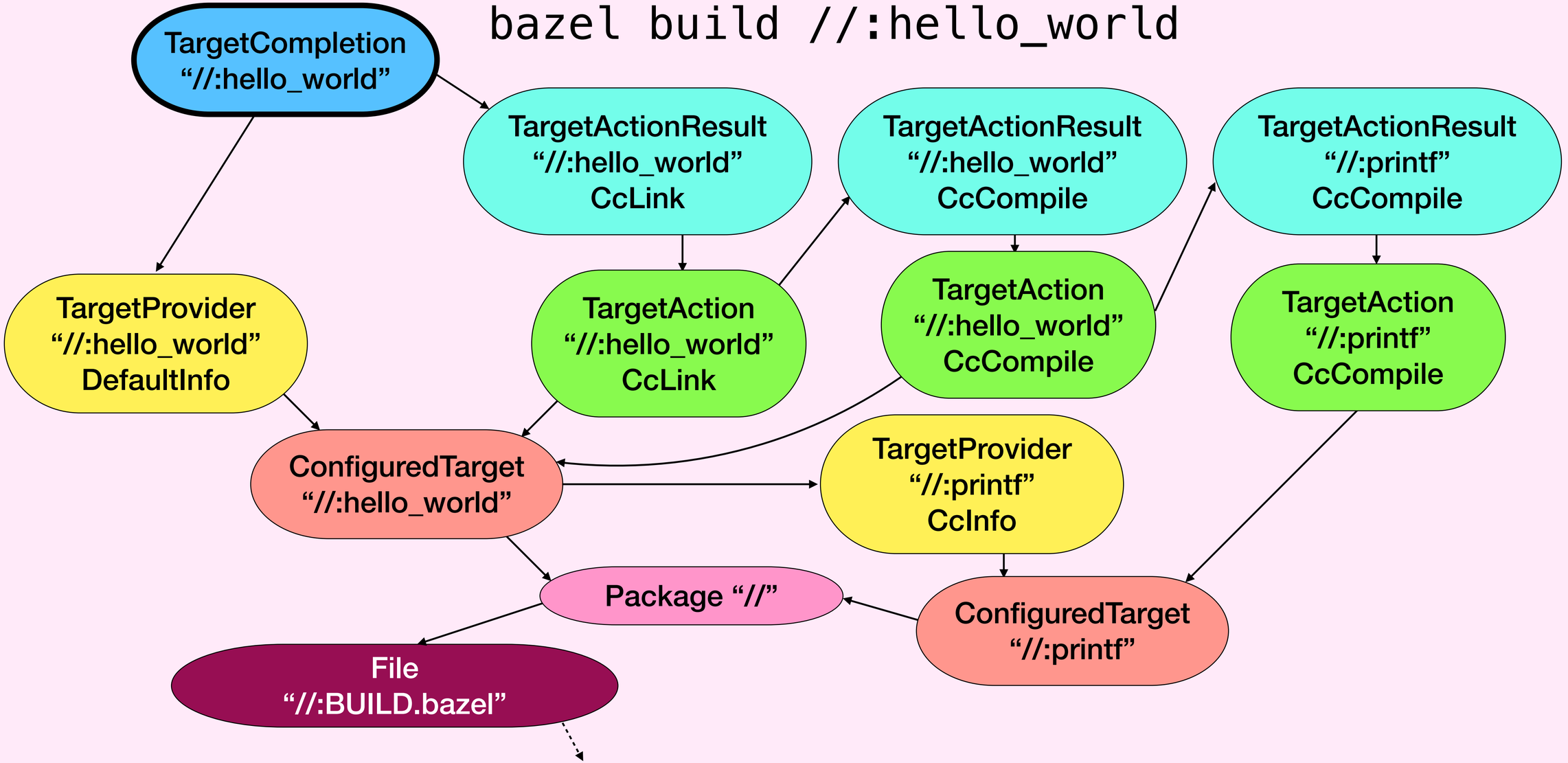
- Merely a way to assign user-defined tags to objects in Object Store.
  - Think of it as **cryptographically signed Git tags**.
  - F : (PublicKey, Identifier) → (ObjectReference, Timestamp, Signature).
  - Need a new namespace/table? Just generate a new key pair.
  - Simple authentication story: only holders of the private key may write.
  - Action Cache: store ActionResult in Object Store, and point a tag to it.

bonanza_builder

Bazel Skyframe

- Bazel's framework for driving the build.
  - **SkyKey:** Identifier for a piece of work that needs to be done.
  - **SkyValue:** Value resulting from a piece of work.
  - **SkyFunction:** F : SkyKey → SkyValue.
  - **Dependencies:** SkyFunction accessing SkyValues of other SkyKeys.
    - Do not need to be known up front.
  - **Restarts:** Stop executing a SkyFunction, because of a missing dependency.

Example Skyframe graph (abridged)



Bonanza's 'Skyframe'

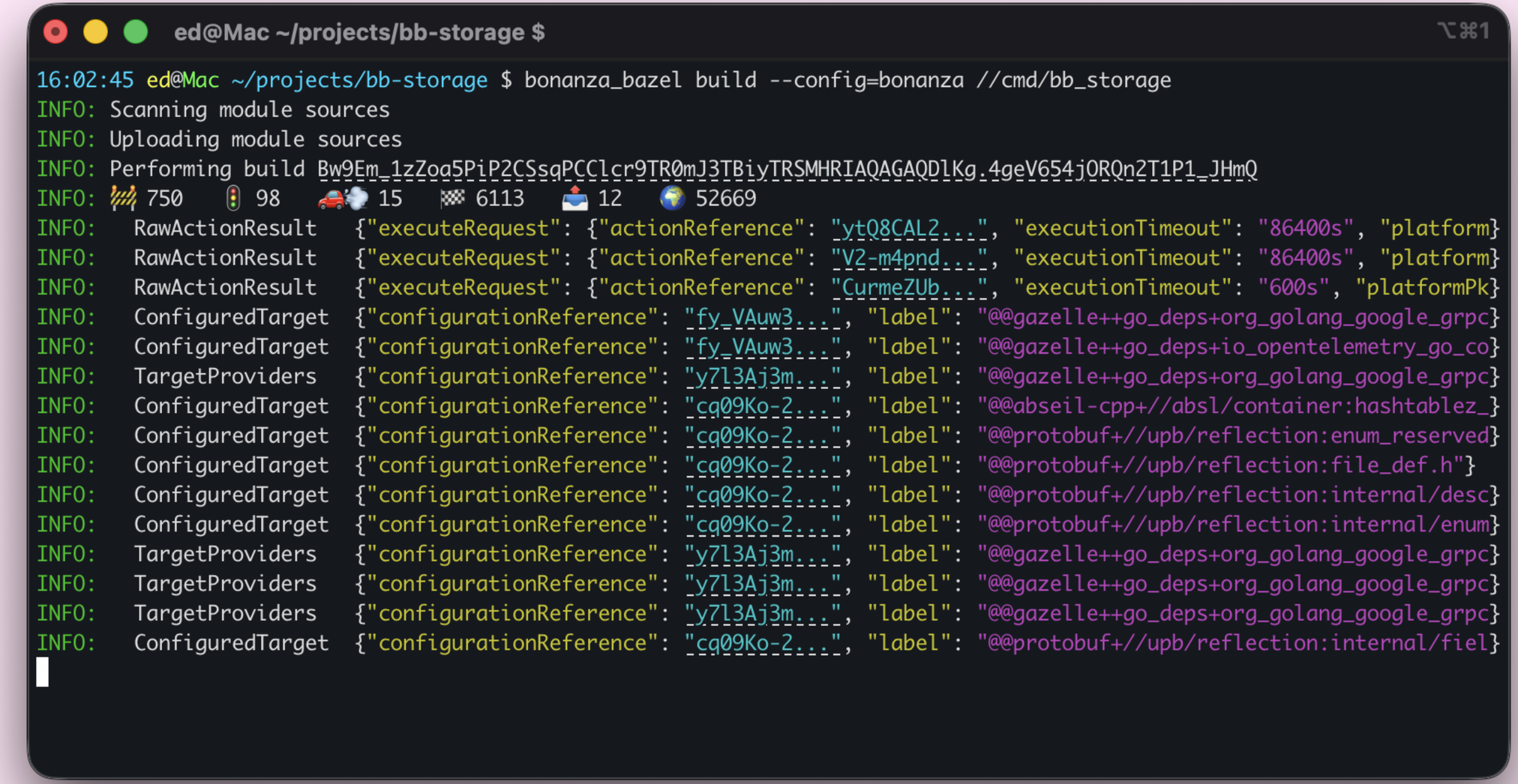
- In Bazel, SkyKeys and SkyValues are Java classes.
- In Bonanza, they are **Merkle trees of Protobuf messages.**
  - Which we can write to our Object Store!
- `bonanza.build/pkg/model/evaluation`
  - Generic logic for queueing SkyKeys and invoking SkyFunctions.
- `bonanza.build/pkg/model/analysis`
  - SkyFunctions that implement Bazel's actual behavior.

Example SkyKey/SkyValue definition

```
message TargetProvider {
  message Key {
    string label = 1;
    bonanza.model.core.DecodableReference configuration_reference = 2
        [(bonanza.model.core.object_format) = {
          proto_list_type_name:
            "bonanza.model.analysis.BuildSettingOverride";
        }];
    string provider_identifier = 3;
  }

  message Value {
    bonanza.model.starlark.Struct.Fields fields = 1;
  }
}
```

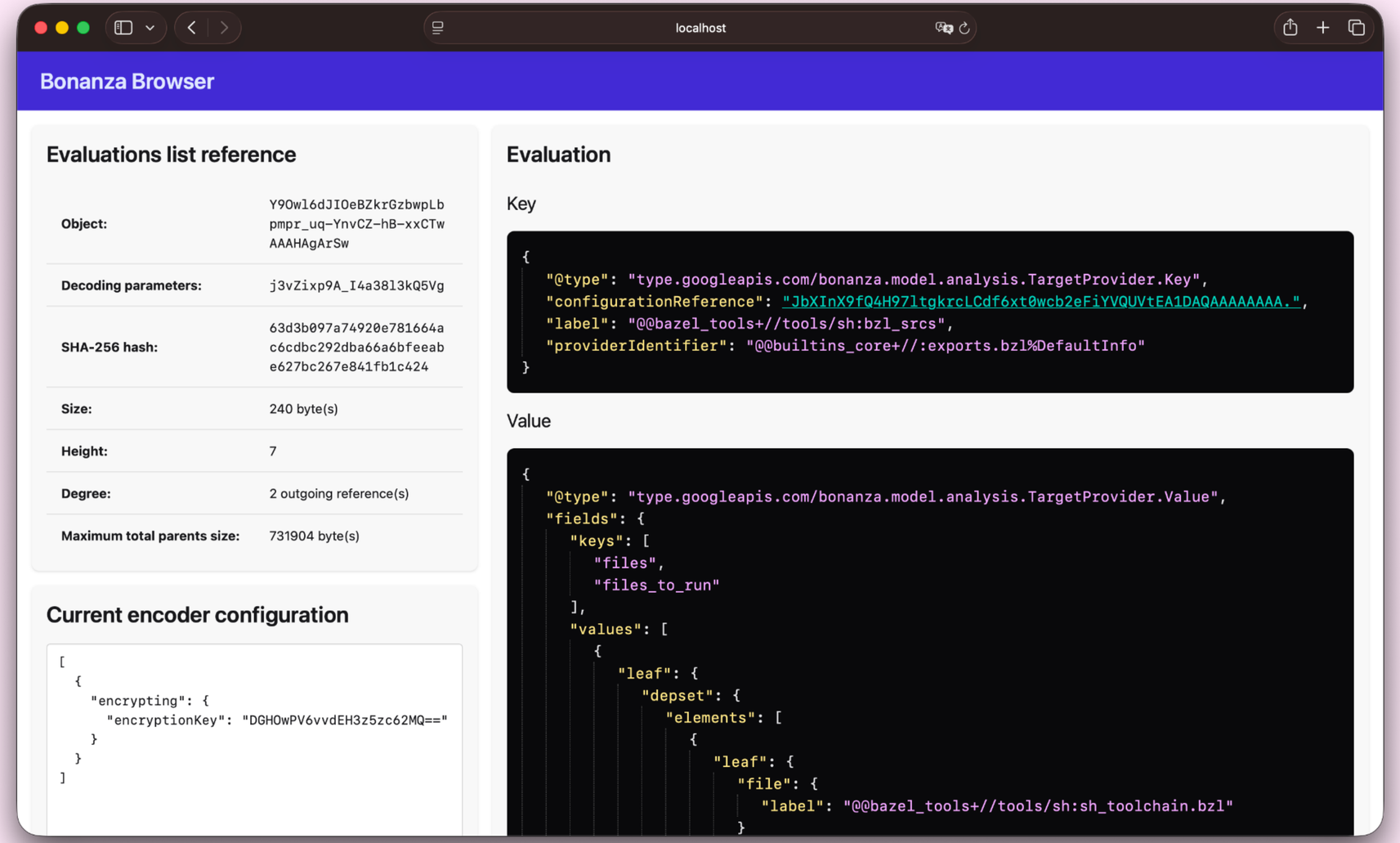
Output while a build is running



Rendering of SkyKeys and SkyValues

```
filegroup(
    name = "bzl_srcs",
    srcs = glob(["*.bzl"]),
    visibility = [...],
)
```

```
def _filegroup_impl(ctx):
    return [DefaultInfo(
        files = depset([...]),
        files_to_run = ...,
    )]
```



Incrementality

- Bazel is designed around incrementality.
  - First run of Bazel is slow.
  - Work performed by subsequent runs is proportional to amount of change.
- Poor performance when switching back and forth between branches.
  - … which will be a constant usage pattern with Bonanza.
- **Bonanza should ideally not use incrementality.**

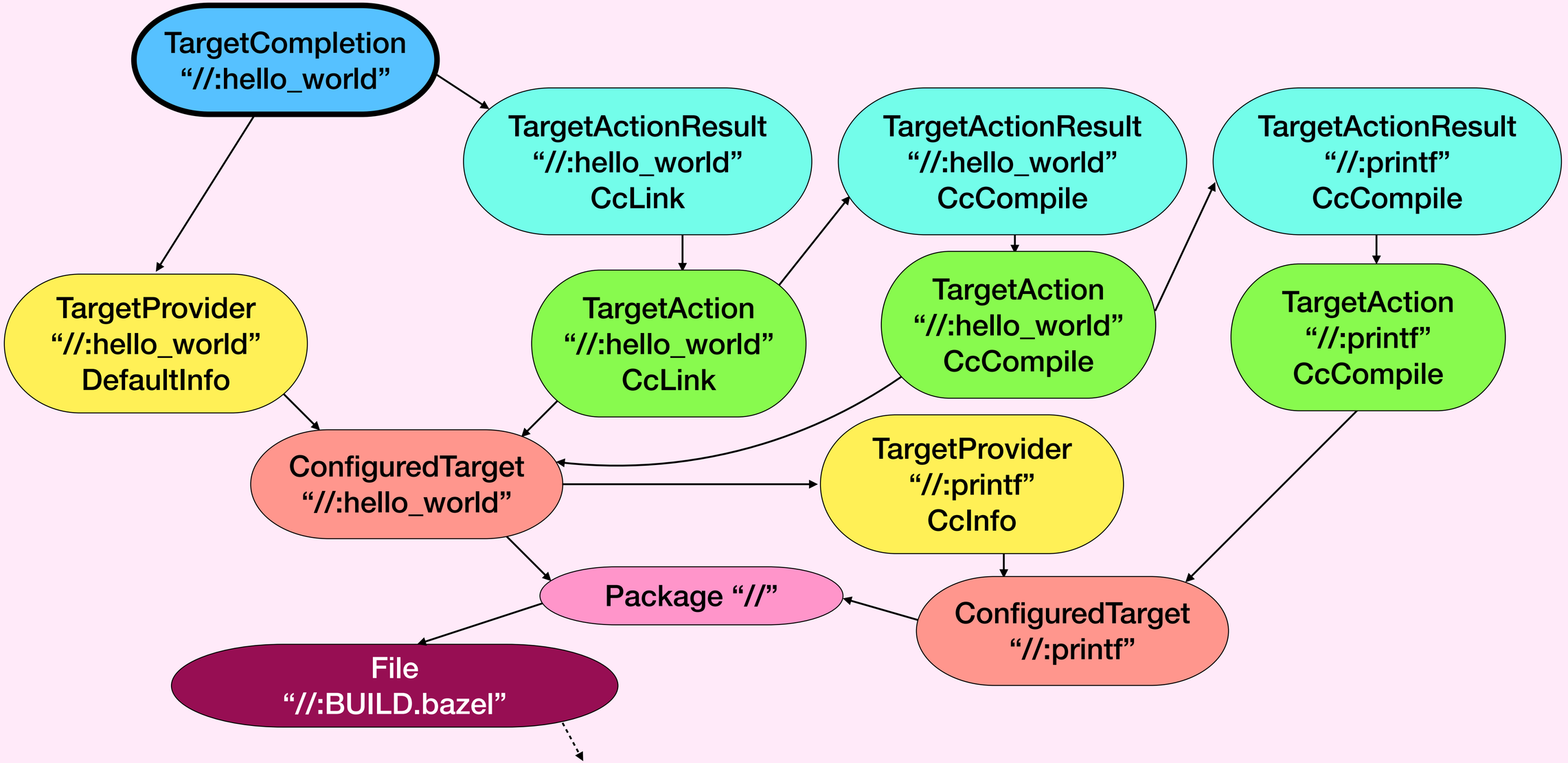
Accessing build results

- Bazel provides access to results via its Build Event Stream.
  - Only contains incremental results.
  - Lots of information it doesn’t contain, like raw REv2 execution details.
  - BES files can become pretty big.
  - BES files can’t easily be diff’ed.
- **I’d like every Bonanza build to yield a full graph of Sky{Key,Value}s.**

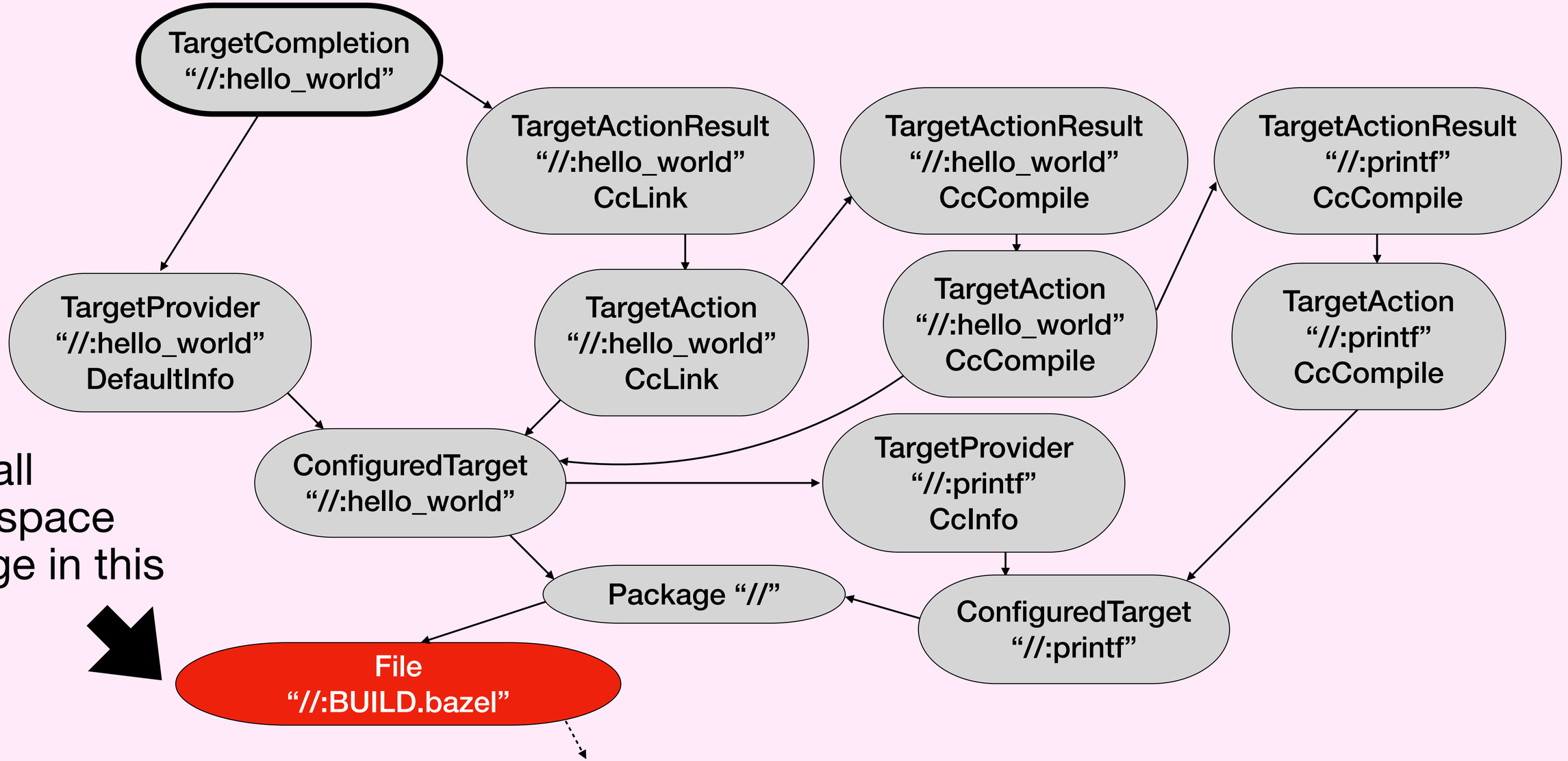
Accessing build results

- Bazel provides access to results via its Build Event Stream.
  - Only contains incremental results.
  - Lots of information it doesn’t contain, like raw REv2 execution details.
  - BES files can become pretty big.
  - BES files can’t easily be diff’ed.
- **I’d like every Bonanza build to yield a Merkle tree of the full graph of Sky{Key,Value}s.**

The shape of the resulting tree?



Storing the graph directly as a Merkle tree



A small whitespace change in this file…

Storing the graph directly as a Merkle tree

